Design Note: Architecture & Domain Model

Candidate: Pruthvi R | CipherSchools 2-Day Engineering Assignment

1. System Overview & User Journey

The platform is engineered as a high-velocity, low-friction practice sandbox guiding learners through a 6-stage cycle: Browse Catalog -> Review Requirements & Constraints -> Draft Solution in Workspace -> Hybrid Evaluation Engine -> Inspect SOLID Radar & Trade-offs -> Refine in Attempt History.

2. Core Domain Model & Class Responsibilities

• Problem: Holds domain specifications, difficulty tags, core functional requirements, expected entity signatures, and benchmark sample implementations.
• Evaluator (Strategy Pattern): Common interface permitting pluggable evaluation strategies (Deterministic, LLM, AST, Linters).
• DeterministicEvaluator: Inspects class counts, interfaces, access modifier encapsulation, and computes baseline SOLID compliance scores.
• AIEvaluator: Synthesizes design patterns (Strategy, State, Observer, Singleton), trade-offs, and actionable refactoring.
• HybridEvaluationPipeline: Coordinates deterministic rules with LLM reasoning, protected by a 4000ms timeout boundary.
• AttemptRepository: Manages timestamped submissions and score progression deltas.

3. Evaluation Philosophy: Deterministic vs. LLM Split

• Deterministic Engine: <15ms execution, 100% offline availability, zero API cost. Validates concrete entity signatures, encapsulation modifiers, and interface inheritance.
• LLM Semantic Engine: Contextual reasoning, design pattern synthesis, architectural trade-offs, and idiomatic refactoring tips.
• Fallback Guarantee: Protected by a Promise.race timeout boundary. If network latency exceeds 4s, the system seamlessly returns deterministic insights without throwing errors.

4. Key Architectural Trade-Offs

• In-App / Client-Side Orchestration vs. Distributed Worker Queues: Eliminates RabbitMQ/Celery complexity for a sub-second interactive MVP while respecting prompt boundaries.
• Code/Pseudocode Editor vs. Drag-and-Drop UML: Real-world engineering interviews require writing concrete classes and interfaces; code-first reflects interview realism.
• Graceful Degradation: Guarantees learners never experience unhandled 500 errors or infinite spinners.